

PRAKTIKUM SISTEM OPERASI

MODUL VII
SEMAPHORE



Oleh:

Haza Zaidan Zidna Fann

2311104056

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

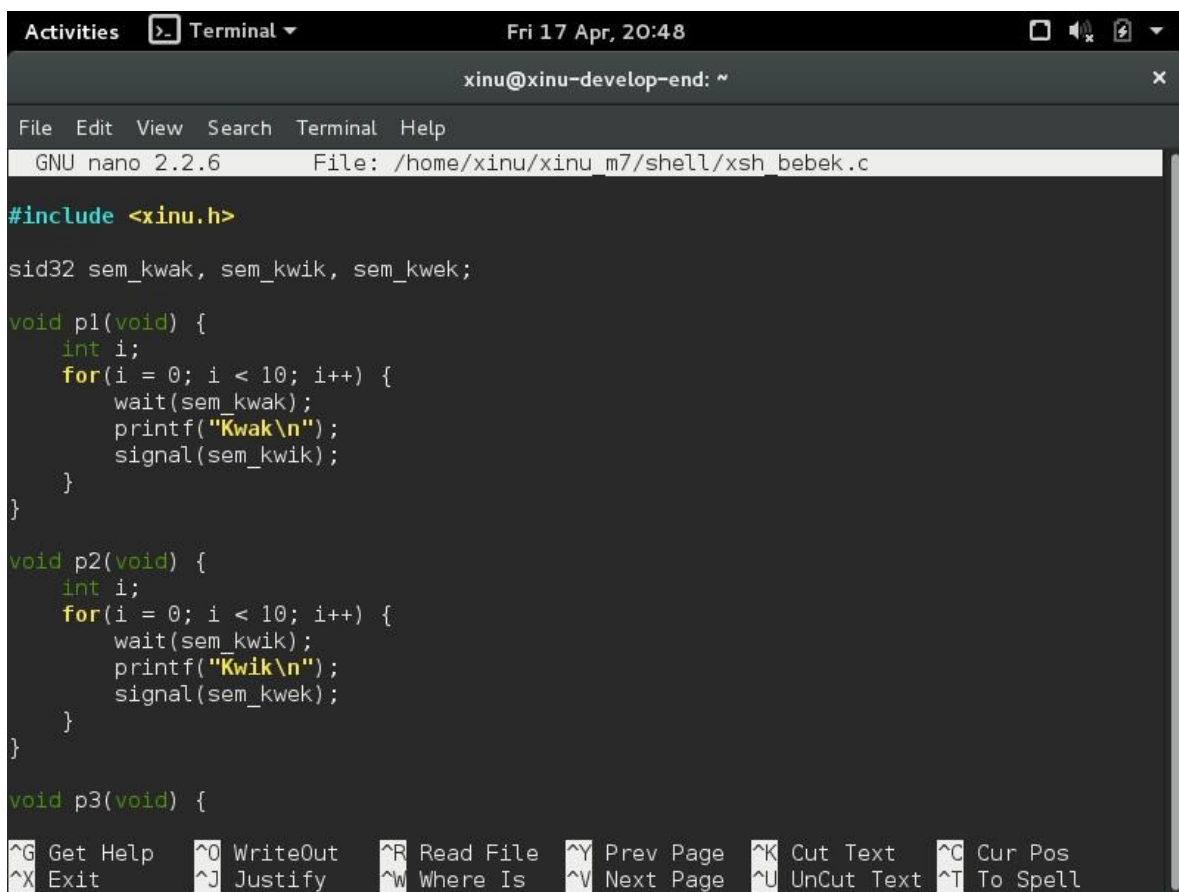
UNIVERSITAS TELKOM

2026

JURNAL

1. Gunakan tiga semaphore untuk mengatur sinkronisasi proses P1, P2, dan P3 agar mencetak urutan "kwak", "kwik", "kwek" secara berulang dan bergantian.

Program ini merupakan implementasi sinkronisasi proses menggunakan semaphore pada sistem operasi Xinu, di mana terdapat tiga proses konkuren yaitu P1, P2, dan P3 yang masing-masing bertugas mencetak output "Kwak", "Kwik", dan "Kwek" secara berurutan dan berulang. Untuk memastikan urutan eksekusi tetap terjaga meskipun proses berjalan secara paralel, digunakan tiga semaphore yaitu sem_kwak, sem_kwik, dan sem_kwek sebagai pengatur giliran. Pada awalnya, sem_kwak diinisialisasi dengan nilai 1 sehingga proses P1 dapat langsung berjalan, sedangkan sem_kwik dan sem_kwek bernilai 0 sehingga P2 dan P3 harus menunggu. Setiap proses melakukan operasi wait() sebelum mencetak output untuk memastikan bahwa proses tersebut hanya berjalan ketika semaphore-nya bernilai positif, kemudian setelah mencetak, proses tersebut akan memanggil signal() untuk mengaktifkan proses berikutnya sesuai urutan, yaitu P1 ke P2, P2 ke P3, dan P3 kembali ke P1 sehingga membentuk siklus berulang. Proses-proses ini dibuat menggunakan fungsi create() dan dijalankan secara konkuren dengan resume(), sementara sleepms() digunakan untuk memberi waktu agar semua proses selesai dieksekusi sebelum semaphore dihapus. Output yang dihasilkan akan selalu konsisten yaitu "Kwak", "Kwik", "Kwek" secara berulang sebanyak 10 kali sesuai dengan loop, yang menunjukkan bahwa penggunaan semaphore berhasil menghindari race condition dan memastikan sinkronisasi antar proses berjalan dengan benar.



```
Activities Terminal Fri 17 Apr, 20:48
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help
GNU nano 2.2.6 File: /home/xinu/xinu_m7/shell/xsh_bebek.c

#include <xinu.h>

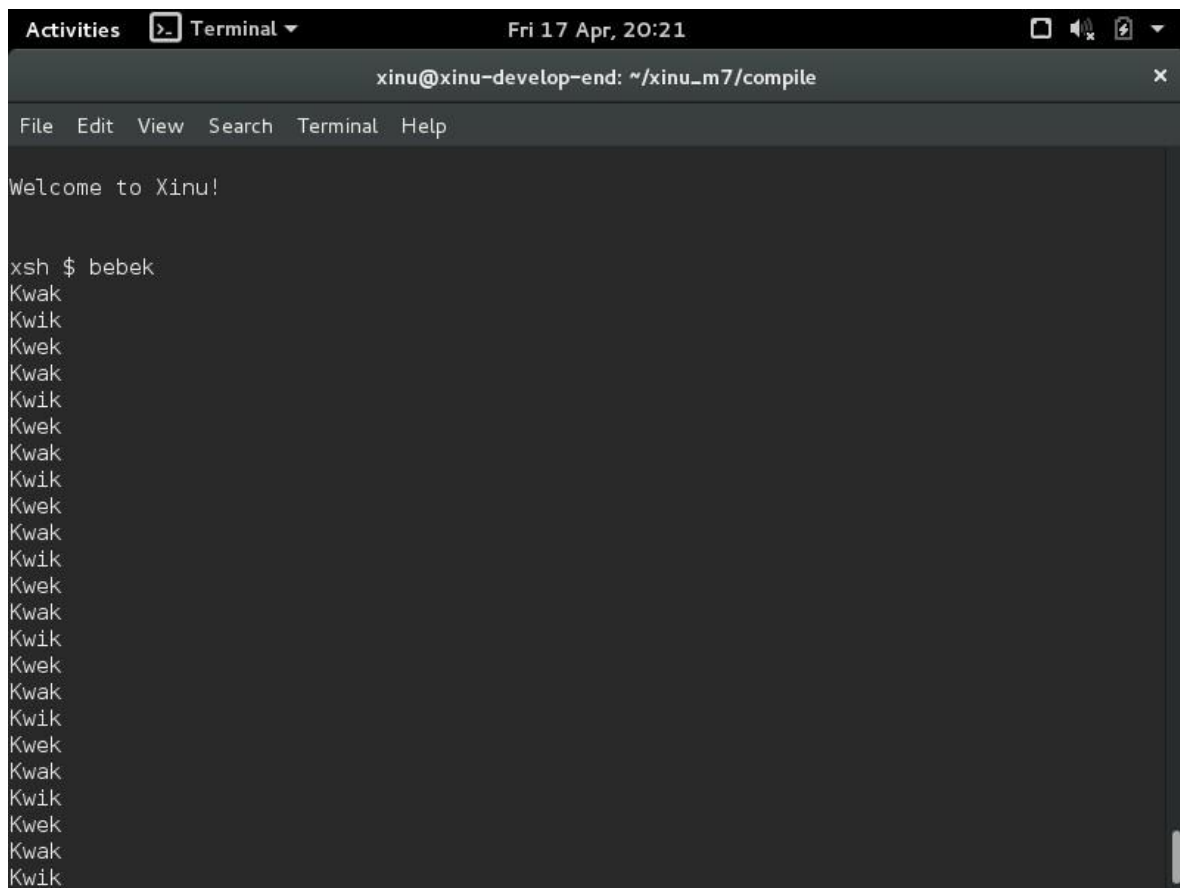
sid32 sem_kwak, sem_kwik, sem_kwek;

void p1(void) {
    int i;
    for(i = 0; i < 10; i++) {
        wait(sem_kwak);
        printf("Kwak\n");
        signal(sem_kwik);
    }
}

void p2(void) {
    int i;
    for(i = 0; i < 10; i++) {
        wait(sem_kwik);
        printf("Kwik\n");
        signal(sem_kwek);
    }
}

void p3(void) {
    int i;
    for(i = 0; i < 10; i++) {
        wait(sem_kwek);
        printf("Kwek\n");
        signal(sem_kwak);
    }
}

int main(void) {
    create(p1, 0);
    create(p2, 0);
    create(p3, 0);
    resume(p1);
    resume(p2);
    resume(p3);
    sleepms(1000);
    return 0;
}
```



```
Activities Terminal Fri 17 Apr, 20:21
xinu@xinu-develop-end: ~/xinu_m7/compile
File Edit View Search Terminal Help

Welcome to Xinu!

xsh $ bebek
Kwak
Kwik
Kwek
Kwak
Kwik
Kwek
Kwak
Kwik
Kwek
Kwak
Kwik
Kwek
Kwak
Kwik
Kwek
Kwak
Kwik
Kwek
Kwak
Kwik
```

```
#include <xinu.h>
```

```
/* Semaphore global: mengatur urutan Kwak → Kwik → Kwek */
```

```
sid32 sem_kwak, sem_kwik, sem_kwek;
```

```
/* Proses P1: cetak "Kwak" */
```

```
void p1(void) {
```

```
    int i;
```

```
    for (i = 0; i < 10; i++) {
```

```
        wait(sem_kwak);    // tunggu giliran Kwak
```

```
        printf("Kwak\n");
```

```
        signal(sem_kwik);  // beri giliran ke Kwik
```

```
    }
```

```
}
```

```
/* Proses P2: cetak "Kwik" */
```

```

void p2(void) {
    int i;
    for (i = 0; i < 10; i++) {
        wait(sem_kwik);    // tunggu giliran Kwik
        printf("Kwik\n");
        signal(sem_kwek);  // beri giliran ke Kwek
    }
}

```

/* Proses P3: cetak "Kwek" */

```

void p3(void) {
    int i;
    for (i = 0; i < 10; i++) {
        wait(sem_kwek);    // tunggu giliran Kwek
        printf("Kwek\n");
        signal(sem_kwak);  // balik ke Kwak
    }
}

```

/* Shell command */

```

shellcmd xsh_bebek(int32 nargs, char *args[]) {

```

/* Inisialisasi semaphore */

```

sem_kwak = semcreate(1); // Kwak jalan dulu

```

```

sem_kwik = semcreate(0); // nunggu

```

```

sem_kwek = semcreate(0); // nunggu

```

/* Jalankan proses */

```

resume(create(p1, 1024, 20, "P1", 0));

```

```

resume(create(p2, 1024, 20, "P2", 0));

```

```

resume(create(p3, 1024, 20, "P3", 0));

```

```
sleepms(1000); // tunggu selesai
```

```
/* Hapus semaphore */
```

```
semdelete(sem_kwak);
```

```
semdelete(sem_kwik);
```

```
semdelete(sem_kwek);
```

```
return 0;
```

```
}
```

2. Sinkronisasikan proses produser dan konsumen menggunakan semaphore untuk memproduksi dan menampilkan bilangan 1 hingga 1000 secara bergantian. Jawaban:

```
#include <xinu.h>
```

```
/* Semaphore dan buffer bersama */
```

```
sid32 sem_empty, sem_full;
```

```
int buffer; // buffer 1 slot
```

```
/* Proses Produser */
```

```
void produser(void) {
```

```
    int i;
```

```
    for (i = 1; i <= 1000; i++) {
```

```
        wait(sem_empty); // tunggu buffer kosong
```

```
        buffer = i; // isi buffer
```

```
        printf("Produser memproduksi nilai %d\n", buffer);
```

```
        signal(sem_full); // kasih tahu ada isi
```

```
    }
```

```
}
```

```
/* Proses Konsumer */
```

```
void konsumer(void) {
```

```
    int i;
```

```
    for (i = 1; i <= 1000; i++) {
```

```
        wait(sem_full); // tunggu ada isi
```

```
        printf("Konsumer menampilkan nilai %d\n", buffer);
```

```
        signal(sem_empty); // kosongin lagi
```

```
    }
```

```
}
```

```
/* Shell command */
```

```
shellcmd xsh_prodcon(int32 nargs, char *args[]) {
```

```

/* Inisialisasi semaphore */
sem_empty = semcreate(1); // buffer kosong
sem_full = semcreate(0); // belum ada isi

/* Jalankan proses */
resume(create(produser, 1024, 20, "Produser", 0));
resume(create(konsumer, 1024, 20, "Konsumer", 0));

sleepms(2000); // tunggu selesai

/* Hapus semaphore */
semdelete(sem_empty);
semdelete(sem_full);

return 0;
}

```

```

Activities  Terminal  Fri 17 Apr, 20:51
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help
GNU nano 2.2.6 File: /home/xinu/xinu_m7/shell/xsh_prodcon.c

#include <xinu.h>

sid32 sem_empty, sem_full;
int buffer;

void produser(void) {
    int i;
    for(i = 1; i <= 1000; i++) {
        wait(sem_empty);
        buffer = i;
        printf("Produser memproduksi nilai %d\n", buffer);
        signal(sem_full);
    }
}

void konsumer(void) {
    int i;
    for(i = 1; i <= 1000; i++) {
        wait(sem_full);
        printf("Konsumer menampilkan nilai %d\n", buffer);
        signal(sem_empty);
    }
}

```

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
 ^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell

```
Activities Terminal Fri 17 Apr, 20:26 xinu@xinu-develop-end: ~/xinu_m7/compile
File Edit View Search Terminal Help
xsh $ prodcon
Produser memproduksi nilai 1
Konsumer menampilkan nilai 1
Produser memproduksi nilai 2
Konsumer menampilkan nilai 2
Produser memproduksi nilai 3
Konsumer menampilkan nilai 3
Produser memproduksi nilai 4
Konsumer menampilkan nilai 4
Produser memproduksi nilai 5
Konsumer menampilkan nilai 5
Produser memproduksi nilai 6
Konsumer menampilkan nilai 6
Produser memproduksi nilai 7
Konsumer menampilkan nilai 7
Produser memproduksi nilai 8
Konsumer menampilkan nilai 8
Produser memproduksi nilai 9
Konsumer menampilkan nilai 9
Produser memproduksi nilai 10
Konsumer menampilkan nilai 10
Produser memproduksi nilai 11
Konsumer menampilkan nilai 11
Produser memproduksi nilai 12
Konsumer menampilkan nilai 12
Produser memproduksi nilai 13
Konsumer menampilkan nilai 13
Produser memproduksi nilai 14
```

```
Activities Terminal Fri 17 Apr, 20:26 xinu@xinu-develop-end: ~/xinu_m7/compile
File Edit View Search Terminal Help
Produser mempKonsumer menampilkan nilai 987
Konsumer menampilkan roduksi nilai 987
Produser memproduksi nilai 988
Prnilai 987
Konsumer menampilkan nilai 989
Konsumer meoduser memproduksi nilai 989
Produser memproduksi ninampilkan nilai 989
Konsumer menampilkan nilai 990
Klai 990
Produser memproduksi nilai 991
Produser memponsumer menampilkan nilai 990
Konsumer menampilkan nroduksi nilai 992
Produser memproduksi nilai 993
Proilai 992
Konsumer menampilkan nilai 994
Konsumer meduser memproduksi nilai 994
Produser memproduksi nilainampilkan nilai 994
Konsumer menampilkan nilai 995
Kon 995
Produser memproduksi nilai 996
Produser memproduksiuser menampilkan nilai 995
Konsumer menampilkan nilasi nilai 997
Produser memproduksi nilai 998
Produseri 997
memproduksi nilai 999
Produser memproduksi nilai 1000
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
```

Program ini merupakan implementasi masalah klasik produser-konsumer menggunakan semaphore pada sistem operasi Xinu, di mana terdapat dua proses yang berjalan secara konkuren yaitu produser yang bertugas menghasilkan data berupa bilangan dari 1 hingga 1000 dan menyimpannya ke dalam buffer tunggal, serta konsumen yang bertugas mengambil dan menampilkan data tersebut. Untuk menghindari kondisi race condition dan memastikan sinkronisasi berjalan dengan benar, digunakan dua semaphore yaitu `sem_empty` dan `sem_full`. Semaphore `sem_empty` digunakan untuk menandakan bahwa buffer dalam keadaan kosong sehingga produser boleh mengisi, sedangkan `sem_full` menandakan bahwa buffer telah berisi data dan siap dibaca oleh konsumen. Pada awalnya, `sem_empty` diinisialisasi dengan nilai 1 (buffer kosong) dan `sem_full` bernilai 0 (belum ada data). Proses produser akan melakukan `wait(sem_empty)` sebelum mengisi buffer agar tidak menimpa data yang belum dibaca, kemudian setelah mengisi buffer dan mencetak output, produser memanggil `signal(sem_full)` untuk memberi tahu konsumen bahwa data siap diambil. Sebaliknya, proses konsumen akan melakukan `wait(sem_full)` untuk menunggu data tersedia, kemudian membaca dan menampilkan nilai dari buffer, lalu memanggil `signal(sem_empty)` untuk memberi tahu produser bahwa buffer kembali kosong. Kedua proses dibuat menggunakan fungsi `create()` dan dijalankan secara paralel dengan `resume()`, serta diberi waktu eksekusi menggunakan `sleepms()` sebelum semaphore dihapus. Output yang dihasilkan akan berupa urutan bergantian antara produser dan konsumen, seperti "Produser memproduksi nilai 1" diikuti "Konsumer menampilkan nilai 1", dan seterusnya hingga 1000, yang menunjukkan bahwa sinkronisasi antar proses berjalan dengan benar tanpa konflik akses data.

Kesimpulan

dari kedua program yang telah dijalankan menunjukkan bahwa penggunaan semaphore sangat efektif dalam mengatur sinkronisasi antar proses yang berjalan secara konkuren pada sistem operasi. Pada program pertama (Kwak-Kwik-Kwek), meskipun terdapat tiga proses yang berjalan secara paralel, output yang dihasilkan tetap teratur secara konsisten yaitu "Kwak", "Kwik", dan "Kwek" secara berulang tanpa terjadi pertukaran urutan atau tabrakan eksekusi, yang membuktikan bahwa semaphore berhasil mengontrol giliran eksekusi setiap proses dengan tepat. Sementara itu, pada program kedua (produser-konsumer), terlihat bahwa proses produser dan konsumen dapat bekerja secara bergantian dengan rapi, di mana setiap data yang diproduksi langsung dikonsumsi tanpa ada data yang terlewat, tertimpa, atau dibaca dua kali, sehingga menunjukkan bahwa mekanisme semaphore mampu menjaga konsistensi data pada shared resource (buffer). Kedua output tersebut secara keseluruhan menegaskan bahwa tanpa adanya semaphore, proses-proses konkuren berpotensi mengalami race condition yang menyebabkan output tidak teratur atau bahkan kesalahan data, namun dengan penerapan semaphore, urutan eksekusi dan integritas data dapat dijaga dengan baik. Dengan demikian, dapat disimpulkan bahwa semaphore merupakan solusi penting dalam pengendalian sinkronisasi dan koordinasi antar proses dalam sistem operasi, khususnya ketika beberapa proses mengakses sumber daya yang sama secara bersamaan.